

1 Abstract multiplicative differential calculus

Source: PrimeTensor/Tensor/Calculus.lean

What this file establishes

Nothing, in the sense of proving: the file contains no theorem. It fixes vocabulary — points, scalar fields, vector fields — and defines the three vector-calculus operators the fluid layer will need, gradient, divergence and Laplacian, on top of an abstract carrier of directional derivatives.

The module header states the reason for the abstraction: it deliberately separates the differential *identities* from the eventual analytic construction of a derivative on completed prime barcodes. A Differential here is a bare family of operators with no axioms, so every identity proved about these three operators downstream will be an identity about folds and components, provable before any analysis exists and true of whatever concrete derivative is eventually supplied.

What is worth reading closely is that the operators are *products*, not sums — Design note 1.6 — and that the folds require no unit, which is a consequence of the decision in PrimeTensor/Depth.lean that nothing starts at zero — Remark 1.7.

1.1 Fields

Definition 1.1 (Point, ScalarField and VectorField). For a type X of coordinate values, a type A of field values and a dimension d ,

$$\begin{aligned}\text{Point}(X, d) &\equiv \text{Axis}(d) \rightarrow X, \\ \text{ScalarField}(X, A, d) &\equiv \text{Point}(X, d) \rightarrow A, \\ \text{VectorField}(X, A, d) &\equiv \text{Point}(X, d) \rightarrow \text{Tensor.Vector}(A, d).\end{aligned}$$

```
-- A point in a positive-dimensional coordinate space. -/
abbrev Point (X : Type) (dim : Depth) := Axis dim → X

-- Scalar field. -/
abbrev ScalarField (X A : Type) (dim : Depth) := Point X dim → A

-- Vector field. -/
abbrev VectorField (X A : Type) (dim : Depth) := Point X dim → Tensor.Vector A dim
```

All three are *abbrevs*, so they unfold on sight and carry no wrapping. The coordinate type X receives no structure anywhere in the file: a point is a tuple of values indexed by an axis, and nothing may be done with it but feed it to a field. By PrimeTensor/Depth.lean the axis is nonempty, so a point always has at least one coordinate.

1.2 The carrier of derivatives

Definition 1.2 (Differential). $\text{Differential}(X, A, d)$ is the structure with the single field

$$d : \text{Axis}(d) \rightarrow \text{ScalarField}(X, A, d) \rightarrow \text{ScalarField}(X, A, d),$$

assigning to each coordinate direction an operator on scalar fields.

```
structure Differential (X A : Type) (dim : Depth) where
  d : Axis dim → ScalarField X A dim → ScalarField X A dim
```

Remark 1.3 (No axioms whatever). The structure has one field and no laws. A Differential is *any* assignment of an endomorphism of the scalar-field space to each axis: nothing requires it to be a morphism of the value carrier, to satisfy a product rule, to commute with itself across directions, or to bear any relation to HasMulDerivativeAt of PrimeTensor/Analysis/Derivative.lean. The two notions of derivative in the development are, at this point, entirely unconnected, and the header says that is deliberate.

The consequence is a division of labour. Everything provable about Definition 1.4 and the two below is provable by unfolding folds and components, with no analysis and no hypotheses — and so holds for whichever concrete Differential is eventually constructed. The price is that nothing provable here has analytic content.

1.3 The three operators

Definition 1.4 (Differential.gradient). $\text{gradient}(D, f)$ is the vector field whose i -th component at x is $D.d\ i\ f\ x$.

The gradient imposes no structure on A at all — it only repackages the family of directional derivatives as a rank-one tensor in the sense of PrimeTensor/Tensor/Basic.lean.

Definition 1.5 (Differential.divergence and laplacian). For A carrying a multiplication,

$$\begin{aligned}\text{divergence}(D, u)(x) &\equiv \text{fold}((\cdot), d, i \mapsto D.d\ i\ (y \mapsto (u\ y).\text{component}\ i)\ x), \\ \text{laplacian}(D, f)(x) &\equiv \text{fold}((\cdot), d, i \mapsto D.d\ i\ (D.d\ i\ f)\ x).\end{aligned}$$

```
-- Multiplicative divergence: product of diagonal directional derivatives. -/
def divergence {X A : Type} [Mul A] {dim : Depth}
  (D : Differential X A dim) (u : VectorField X A dim) : ScalarField X A dim :=
  fun x =>
    Axis.fold (· * ·) dim (fun i =>
      D.d i (fun y => (u y).component i) x)
```

Design note 1.6 (Products where the classical operators have sums). Both operators fold with multiplication. Classically

$$\text{div } u = \sum_i \partial_i u_i, \quad \Delta f = \sum_i \partial_i \partial_i f,$$

and the sums here are products. This is the same substitution the whole development makes, and it reads correctly in the logarithmic chart of PrimeTensor/Analysis/Derivative.lean: since log turns a product into a sum, the multiplicative divergence of u is the ordinary divergence of the log-transformed field, and likewise for the Laplacian. Nothing is lost and nothing additive is introduced.

Two features of the classical operators survive unchanged. The divergence contracts a vector field to a scalar by pairing the i -th direction with the i -th component — the diagonal, as the docstring says. And the Laplacian takes only *same-axis* second derivatives, $D_i D_i f$: no mixed partial $D_i D_j f$ with $i \neq j$ appears in this file, and no commutation of mixed partials is available to be assumed, since Remark 1.3 leaves the operators unrelated.

Remark 1.7 (The fold needs no unit, and that is the point of positive depth). The two definitions require `[Mul A]` and nothing else. In particular they do not require `[One A]`, even though a product over an index set normally needs a unit for the empty case.

There is no empty case. `fold` of `PrimeTensor/Depth.lean` was defined with no identity argument precisely because `Axis(d)` always has at least one coordinate, `Depth` having no zeroth constructor. So the divergence of a vector field over a one-dimensional axis is its single directional derivative, not that derivative multiplied by an empty product.

This is the concrete payoff of the founding decision, and it matters more here than a convenience: the eventual value carrier is `MulReal`, which does have a unit, but the definition is stated for an arbitrary type with a multiplication, and that generality is available only because no unit is needed.

Remark 1.8 (Order and bracketing are not shown to be irrelevant). `fold` associates to the right and visits the axis in the order `first`, then the successors, and the definitions inherit that. Since `A` carries only `[Mul A]`, with neither associativity nor commutativity assumed, the value of a divergence genuinely depends on the bracketing and on the enumeration of the axis. For a carrier satisfying `IsMulCarrier` of `PrimeTensor/Structure.lean` both dependencies vanish, but this file neither assumes that nor proves the independence. Any later identity that reorders a fold will have to establish it.

Remark 1.9 (What is absent). There is no curl, no second-order mixed operator, and no theorem of any kind — no linearity, no product rule, no identity relating divergence to gradient, and no analogue of $\text{div grad} = \Delta$, which in this notation would say that the divergence of the gradient of f is the Laplacian of f and is in fact true by unfolding, the two folds having the same summand. It is not stated. Nor is any `Differential` constructed: the structure has no instance anywhere in this file, so the three operators are so far defined and never applied.

Summary of the correspondence

Lean declaration	Kind	Here
<code>Point</code>	abbrev	Def. 1.1
<code>ScalarField</code>	abbrev	Def. 1.1
<code>VectorField</code>	abbrev	Def. 1.1
<code>Differential</code>	structure	Def. 1.2
<code>Differential.gradient</code>	definition	Def. 1.4
<code>Differential.divergence</code>	definition	Def. 1.5
<code>Differential.laplacian</code>	definition	Def. 1.5

Every declaration of `PrimeTensor/Tensor/Calculus.lean` appears above, together with the single field `d` of `Differential`. The file contains no theorem, and so no `sorry`, no axiom, and no unproved named proposition. Design note 1.6 and Remarks 1.3, 1.7, 1.8 and 1.9 are stated for the reader; the logarithmic reading of the two folds is a gloss and is not formalised.